מסמך אפיון דרישות תוכנה (SRS)

Smart Campus Assistant

<u>עבור אלעד מערכות מידע</u>

נכתב ע״י יוסף שגב

מרץ 2026

1. סקירת המערכת (System Overview)

Smart Campus Assistant היא אפליקציית Web קלת-משקל שמטרתה לספק לסטודנטים מענה מיידי לשאלות הקשורות לחיי הקמפוס באמצעות מודל בינה מלאכותית.

סטודנטים נדרשים לעיתים קרובות למידע כגון לוחות מבחנים, מיקום כיתות, שעות קבלה של מרצים, מידע אדמיניסטרטיבי או פתרון בעיות טכניות. במקום לפנות למספר מערכות שונות או למזכירות, המערכת מאפשרת לשאול שאלה בשפה טבעית ולקבל תשובה מיידית.

המערכת מבוססת על ארכיטקטורת Web הכוללת שלושה רכיבים מרכזיים:

- **Frontend (React Web Application)** – ממשק משתמש בדפדפן המאפשר לסטודנטים להזין שאלות ולקבל תשובות.
- **Backend (FastAPI)** – שרת המטפל בבקשות המשתמשים, מבצע עיבוד נתונים ומתקשר עם שירות הבינה המלאכותית.
- **AI Processing Service** – שירות AI חיצוני (OpenAI) שמנתח את השאלה ומייצר תשובה.

בנוסף, המערכת משתמשת במסד נתונים **SQLite Database** המכיל מידע קמפוסי מובנה כגון:

- לוחות מבחנים
- קורסים
- מידע על מרצים
- שעות קבלה של מרצים
- מיקומי כיתות
- שאלות נפוצות (FAQ)
- עזרה טכנית

מטרת המערכת היא לשפר את זמינות המידע לסטודנטים ולהפחית עומס על שירותי האדמיניסטרציה.

2. מגבלות מערכת (System Constraints)

המערכת תפעל תחת מספר מגבלות טכנולוגיות ותפעוליות:

- המערכת תשתמש במסד נתונים **SQLite**, ולכן אינה מיועדת בשלב הראשון לניהול כמויות מידע גדולות במיוחד.
- שירות הבינה המלאכותית מבוסס על **External AI API**, ולכן תלוי בזמינות השירות החיצוני.
- זמן התגובה הכולל של המערכת תלוי הן בביצועי שרת ה-Backend והן בזמן העיבוד של שירות ה-AI.
- המערכת מיועדת לשימוש דרך **Web Browser** בלבד ואינה כוללת אפליקציה ייעודית למובייל בשלב זה.

3. תכולת המערכת (System Scope)

המערכת תכלול את הרכיבים הבאים:

- ממשק משתמש Web לשאילת שאלות
- שירות Backend מבוסס Python ו-FastAPI
- אינטגרציה עם שירות AI לעיבוד שפה טבעית
- מסד נתונים SQLite עם מידע קמפוסי מובנה
- מנגנון סיווג שאלות לקטגוריות שונות
- מנגנון Fallback למקרים שבהם המערכת אינה בטוחה בתשובה

המערכת אינה כוללת בשלב זה:

- מערכת ניהול משתמשים מלאה
- אינטגרציה ישירה עם מערכות מידע אקדמיות
- אפליקציה ייעודית למובייל

4. בעלי עניין (Stakeholders)

סטודנטים (Students)
משתמשי הקצה של המערכת. משתמשים במערכת לצורך קבלת מידע על שירותי הקמפוס.

צוות אדמיניסטרציה / IT (Campus IT Staff)
אחראים על תחזוקת המידע במסד הנתונים ועל זמינות המערכת.

הנהלת הקמפוס (Campus Management)
נהנית משיפור השירות לסטודנטים ומהפחתת עומס על מוקדי השירות.

צוות הפיתוח (Development Team)
אחראי על פיתוח, תחזוקה ושיפור המערכת.

5. תרחישי שימוש (Use Cases)

Use Case 1 – בדיקת מועד מבחן

Input: "מתי מתקיים המבחן במבוא למדעי המחשב?"

System Response:
המערכת מזהה כי מדובר בשאלה מסוג **Schedule Inquiry**, שולפת את הנתונים ממסד הנתונים ומחזירה את תאריך המבחן ואת מיקומו.

Use Case 2 – איתור כיתה

Input: "איפה נמצא חדר 204 בבניין מדעים?"

System Response:
המערכת מבצעת שליפה ממסד הנתונים ומחזירה את מיקום הכיתה.

Use Case 3 – מידע אדמיניסטרטיבי

Input: "איך מוציאים אישור לימודים?"

System Response:
המערכת מחזירה את ההנחיות הרלוונטיות מתוך מאגר ה-FAQ.

Use Case 4 – בעיה טכנית

Input: "Why can't I connect to the campus WiFi?"

System Response:
המערכת מזהה את השאלה כ-**Technical Issue** ומחזירה הנחיות לפתרון התקלה.

Use Case 5 – שאלה מחוץ לתחום (Fallback)

Input: שאלה שאינה קשורה לשירותי הקמפוס.

System Response:
המערכת מחזירה הודעה מנומסת:

"מצטערים, איני בטוח בתשובה לשאלה זו. אנא נסה לשאול שאלה הקשורה לשירותי הקמפוס."

Use Case 6 – שעות קבלה של מרצה

Input: "מתי שעות הקבלה של ד"ר כהן?"

System Response:
המערכת שולפת את שעות הקבלה ממסד הנתונים ומחזירה את המידע למשתמש.

Use Case 7 — שאלה שאין לה מידע במסד הנתונים

Input: "מתי מתקיים המבחן בקורס ניתוח מערכות מתקדמות?"

System Response:
המערכת מבצעת חיפוש במסד הנתונים SQLite Database. במידה והמידע אינו קיים במסד הנתונים, המערכת תחזיר הודעה למשתמש כגון:

"לא נמצא מידע לגבי הקורס המבוקש במערכת. ייתכן שהמידע עדיין לא עודכן."

Use Case 8 – שאלה הכוללת מספר נושאים

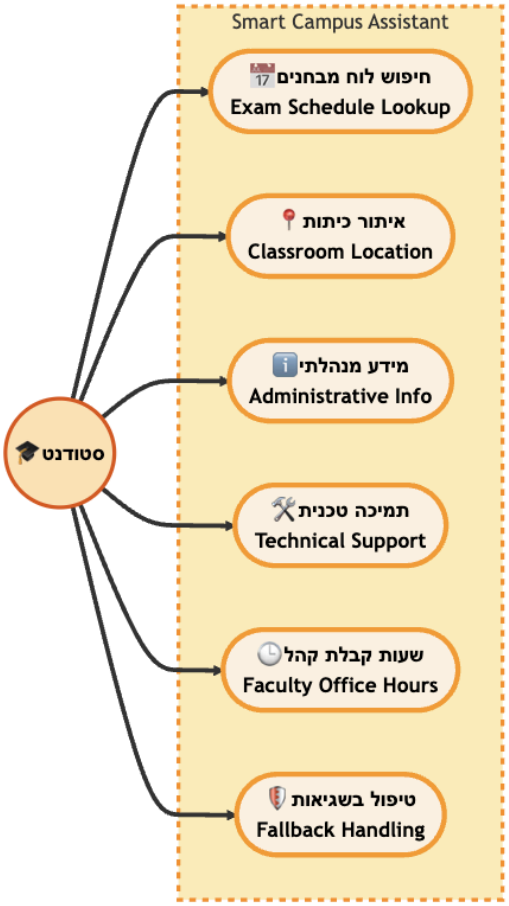
Input: "איפה מתקיים המבחן במבוא למדעי המחשב ומתי הוא מתחיל?"

System Response:
המערכת מזהה כי השאלה כוללת יותר מסוג מידע אחד (מיקום ומועד). שירות ה-AI מבצע **Intent Analysis** ומזהה את שני רכיבי השאלה: מועד המבחן ומיקום המבחן

שרת ה-Backend שולף את הנתונים הרלוונטיים ממסד הנתונים **SQLite Database** ומחזיר תשובה מאוחדת למשתמש. לדוגמה:

"המבחן במבוא למדעי המחשב מתקיים בתאריך 12.07 בשעה 10:00 בחדר 204 בבניין מדעים."

Use Case Diagram



6. תרחיש שימוש מלא (User Scenario- Flow)

תרחיש: איתור חדר בחינה תחת לחץ זמן

- **תיאור המצב:** סטודנט מגיע לקמפוס 10 דקות לפני תחילת מבחן ב"מבוא למדעי המחשב" ואינו זוכר באיזה אולם הבחינה מתקיימת.
- **פעולה:** הסטודנט פותח את אפליקציית Smart Campus Assistant בנייד ומקליד: "איפה המבחן במבוא למדעי המחשב היום?".
- **עיבוד המערכת:**
  1. השרת מקבל את הבקשה ומעביר אותה למודל ה-AI.
  2. ה-AI מזהה את כוונת המשתמש ומסווג את השאלה לקטגוריית "לוח זמנים/מבחנים".
  3. השרת מזהה את הקטגוריה, מחלץ את שם הקורס ("מבוא למדעי המחשב"), וניגש למסד הנתונים (SQLite) לאיתור הרשומה הרלוונטית.
  4. השרת מעביר את המידע הגולמי (שעת המבחן + בניין 72, חדר 214) חזרה ל-AI.
  5. ה-AI מנסח תשובה קצרה ומרגיעה.
- **תוצאה:** הסטודנט מקבל תשובה תוך שתי שניות: "המבחן במבוא למדעי המחשב מתקיים היום בשעה 10:00 בבניין 72, חדר 214. בהצלחה!".

7. דרישות פונקציונליות (Functional Requirements)

FR-1
המערכת תספק REST API endpoint בכתובת: `POST /ask` לקבלת שאלות משתמשים.

FR-2
המערכת תסווג שאלות לאחת משלוש קטגוריות:

- Schedule Inquiry
- General Information
- Technical Issue

FR-3
המערכת תשתמש בשירות **AI API** חיצוני לצורך ניתוח השאלה.

FR-4
המערכת תשלוף מידע רלוונטי מתוך **SQLite Database**.

FR-5
המערכת תחזיר תשובה בפורמט **JSON Response** הכולל:

- **Category:** סיווג לאחת הקטגוריות: Schedule, General, Technical.
- **Language:** השפה שזוהתה (Hebrew/English).
- **Confidence:** רמת וודאות להפעלת מנגנון Fallback במידת הצורך.
- **Answer:** התשובה המנוסחת למשתמש.

FR-6
המערכת תרשום **system logs** של בקשות משתמשים לצורך ניטור ושיפור ביצועי המערכת.

FR-7
המערכת תאפשר טיפול ב-**concurrent requests** ממספר משתמשים במקביל.

FR-8
המערכת תפעיל **Fallback Mechanism** במקרה של חוסר ודאות בתשובת ה-AI.

FR-9
המערכת תספק **Admin Interface** בסיסי עבור צוות IT לצורך:

- עדכון מידע במסד הנתונים
- צפייה ב-logs
  ניטור פעילות המערכת

8. דרישות לא פונקציונליות (Non-Functional Requirements)

Performance

זמן תגובה ממוצע של המערכת יהיה עד **3 שניות** לבקשה.

Availability

שירות ה-Backend יהיה זמין לפחות **uptime 99.9%**.

Security

מפתחות API של שירותי AI ינוהלו באמצעות **Environment Variables (.env)** ולא יישמרו בקוד המקור.

Scalability

המערכת תוכל לטפל בלפחות **requests per minute 100** ללא פגיעה משמעותית בביצועים.

Language Support

שירות ה-AI יתמוך בעיבוד שאלות בשתי שפות:

- Hebrew
- English

המערכת תזהה את שפת השאלה ותייצר תשובה באותה השפה.

Accessibility

ממשק המשתמש (Frontend) יעמוד בתקן הנגישות WCAG 2.1 ברמת AA, כולל תמיכה בקוראי מסך וניווט באמצעות מקלדת עבור סטודנטים בעלי מוגבלויות.

9. מדדי הצלחה (Success Metrics)

הצלחת המערכת תימדד באמצעות המדדים הבאים:

- **דיוק מידע (Accuracy):** ה-AI יספק תשובות נכונות ויסווג נכונה לפחות 90% מהשאלות בסט הבדיקות.
- **ביצועים (Performance):** זמן התגובה (Latency) של המערכת לא יעלה על 3 שניות בממוצע מרגע שליחת השאלה.
- **זמינות (Availability):** זמינות המערכת (Uptime) תעמוד על לפחות 99.9% לאורך זמן.
- **הפחתת עומס (Load Reduction):** הפחתה משמעותית בפניות סטודנטים למוקדי מידע אדמיניסטרטיביים ולמזכירויות הפקולטה.
- **אמינות (Reliability):** מנגנון ה-Fallback יופעל ב-100% מהמקרים בהם המערכת אינה בטוחה בתשובה.
- **יציבות פריסה (Deployment):** המערכת תעלה בצורה תקינה באמצעות פקודת Docker אחת, ללא שגיאות תלות.

10. מקרי קצה (Edge Cases)

עומס בקשות גבוה

במקרה שבו מספר רב של משתמשים שולחים בקשות למערכת בו-זמנית (לדוגמה בתקופת מבחנים), ייתכן עומס על שרת ה-Backend.

במצב כזה, המערכת תאפשר טיפול בבקשות באמצעות **asynchronous processing** ותתמוך ב-**concurrent requests**.

אם מספר הבקשות חורג מהמגבלה המוגדרת, השרת יחזיר תגובת:

`Too Many Requests 429`

קלט ריק או לא תקין

אם המשתמש שולח קלט ריק או ג'יבריש, השרת יחזיר תגובת:

`Bad Request 400`

עם בקשה לנסח את השאלה מחדש.

קלט דו-משמעי (Ambiguity)

אם המשתמש שולח קלט עם חוסר מידע לדוגמה: "מתי המבחן?" (ללא שם קורס).

המערכת תזהה חוסר במידע ותחזיר שאלה מנחה: "באיזה קורס תרצה לבדוק את מועד הבחינה?"

תקלה בשירות ה-AI

אם שירות ה-AI אינו מגיב בתוך **5 seconds timeout**, המערכת תחזיר הודעה מנוהלת למשתמש.

מחוץ לתחום (Fallback)

אם המשתמש שולח קלט מחוץ לתחום אחריות האפליקציה לדוגמה: "איך מכינים פיצה?"

המערכת תזהה זאת ותפעיל Fallback: "מצטערים, איני בטוח בתשובה. אנא שאל שאלה הקשורה לקמפוס"

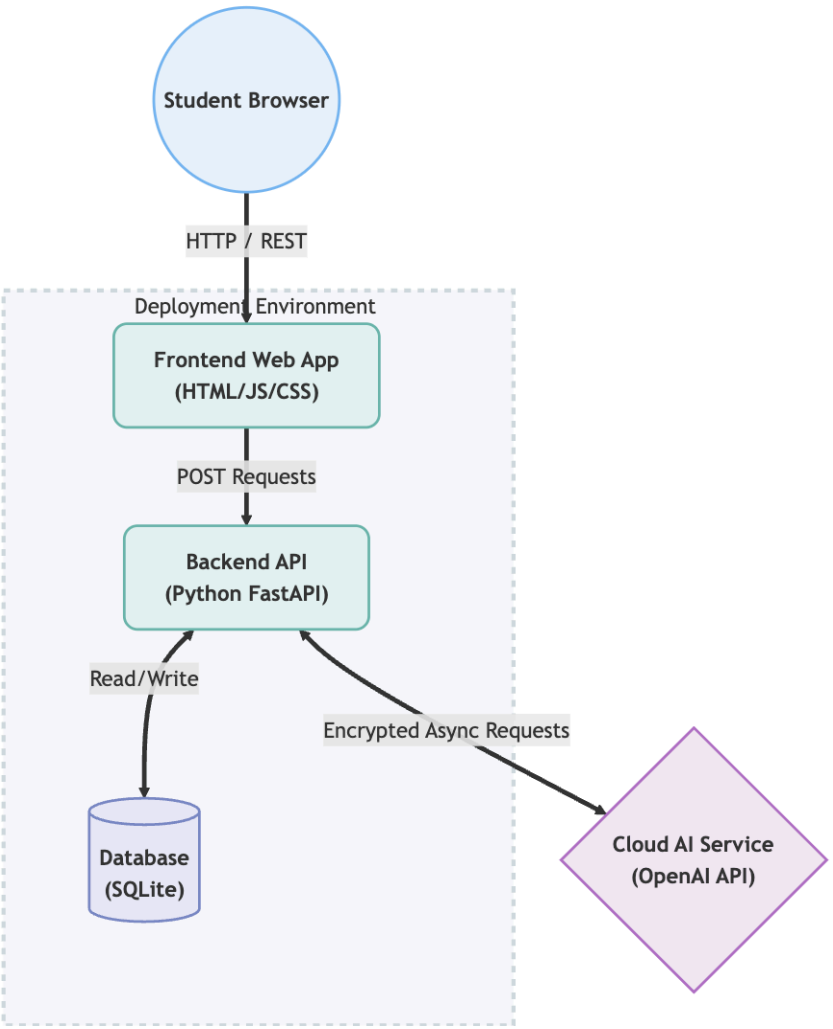
11. ארכיטקטורת מערכת (System Architecture)

המערכת בנויה בארכיטקטורה רב-שכבתית:

1. הסטודנט שולח שאלה דרך ה-Frontend.
2. ה-Frontend שולח בקשת `POST /ask` ל-Backend.
3. ה-Backend שולח בקשה אסינכרונית ל-AI Service.
4. ה-AI Service מחזיר תשובה וקטגוריה.
5. ה-Backend מבצע שאילתה ל-SQLite Database במידת הצורך.
6. ה-Backend מחזיר את התשובה ל-Frontend.
7. ה-Frontend מציג את התוצאה למשתמש.

שרת ה-Backend משמש כשכבת תיווך המרכזית בין ממשק המשתמש, שירות ה-AI ומסד הנתונים.

System Architecture and Deployment Diagram



עיבוד שאלות באמצעות AI Processing

שירות ה-AI משמש לניתוח שאלות המשתמש, סיווגן והפקת תשובות רלוונטיות.

תהליך העיבוד מתבצע לפי השלבים הבאים:

1. User Question
2. POST /ask
3. Backend Processing
4. AI Classification + Response Generation
5. Optional Data Query - SQLite
6. Response to User

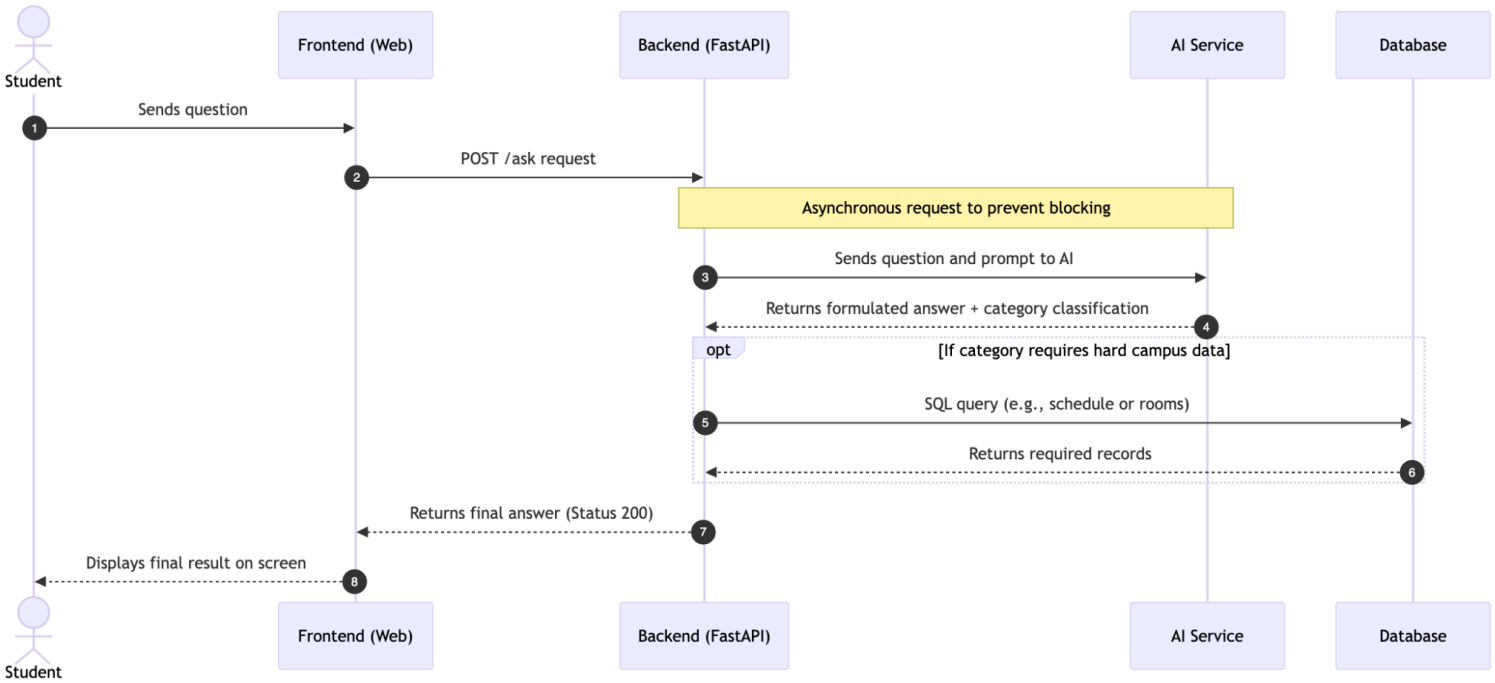
ה-AI אחראי על הפעולות הבאות:

- **Language Detection** – זיהוי שפת השאלה (Hebrew / English)
- **Intent Classification** – סיווג השאלה לקטגוריה
- **Response Generation** – יצירת תשובה ברורה למשתמש

Sequence Diagram

התהליך במערכת מבוסס על ארכיטקטורה רב-שכבתית המנוהלת באופן אסינכרוני כדי למנוע חסימת משתמשים:

- **ניהול הבקשה:** ה- Backend (FastAPI) מקבל את השאלה מה-Frontend ומתעד אותה ב-Logs לצרכי ניטור.
- **עיבוד AI אסינכרוני:** השרת שולח בקשה מוצפנת לשירות ה-OpenAI לצורך סיווג כוונה (Intent) וזיהוי שפה.
- **שליפת נתונים מותנית:** במידה וה-AI מסווג את השאלה ככזו הדורשת נתונים קשיחים (כגון לוחות מבחנים או מיקום כיתות), ה-Backend מבצע שאילתת SQL למסד הנתונים SQLite.
- **גיבוש תשובה:** השרת מאחד את הנתונים הגולמיים מה-DB עם הניסוח של ה-AI ומחזיר אובייקט JSON סופי למשתמש.



12. שלבי הפרויקט (Project Phases)

הפרויקט יפותח במספר שלבים עיקריים:

1. **System Analysis** – הגדרת דרישות המערכת וכתיבת מסמך SRS.
2. **Backend Development** – פיתוח שירות API באמצעות FastAPI.
3. **Frontend Development** – פיתוח ממשק משתמש באמצעות React.
4. **Integration & Testing** – חיבור כל רכיבי המערכת וביצוע בדיקות.
5. **Deployment** – פריסת המערכת והכנת תיעוד.

---

13. הרחבות עתידיות (Future Extensions)

- הוספת מערכת אימות והרשאות (Authentication System)
- אינטגרציה עם מערכות מידע אקדמיות
- שדרוג ל-RAG מתקדם ומסד נתונים וקטורי (Vector DB)
- יישום מטמון סמנטי (Semantic Caching)
- מעבר למסד נתונים PostgreSQL
- הוספת מערכת לניהול מידע (Admin CMS)
- פיתוח אפליקציית מובייל (Mobile Application)